

CTF 7 — Stop Putting Everything on One Screen

Goal: Introduce Angular routing and separate the Student Portal into dedicated pages.

School Request

"Each student deserves a dedicated page."

Today's mission introduces **Angular Router**. Instead of keeping the entire Student Portal on one screen, you'll create separate pages for the student list, student details, and adding a new student.

You'll also practice navigating between pages using Angular routes.

Flag 1 — Prepare Your Mission

Objective: Create a new branch for today's challenge.

Create and switch to a new Git branch.

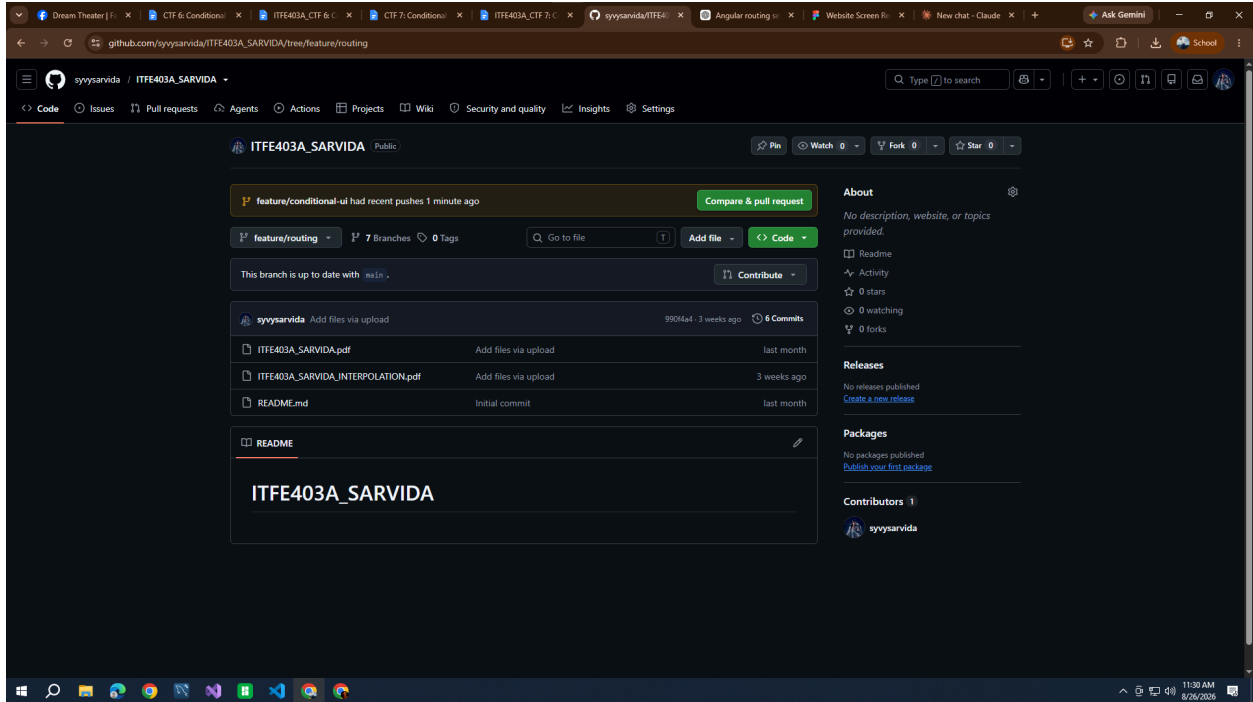
Branch name:

feature/routing



Screenshot:

- The command used to create and switch to the new branch.



Flag 2 — Build the Student List Page

Objective: Create a dedicated page for the student list.

Move your existing student list UI into a separate component/page.

Your page should display:

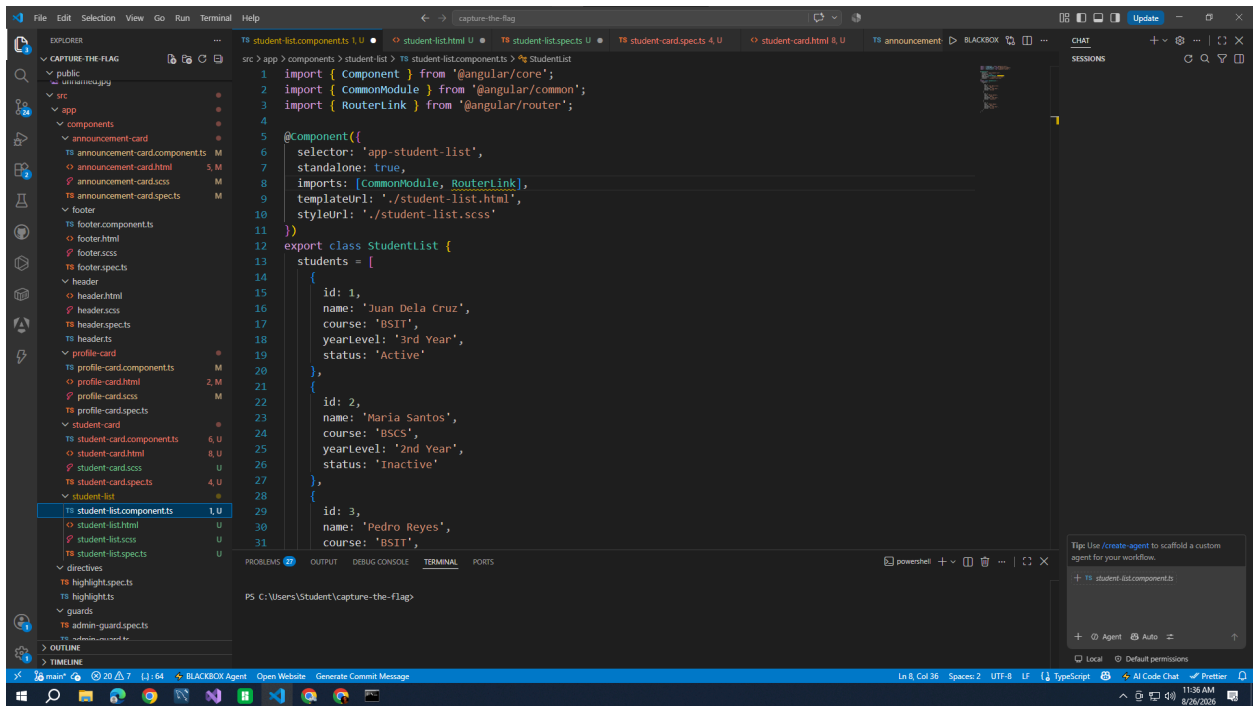
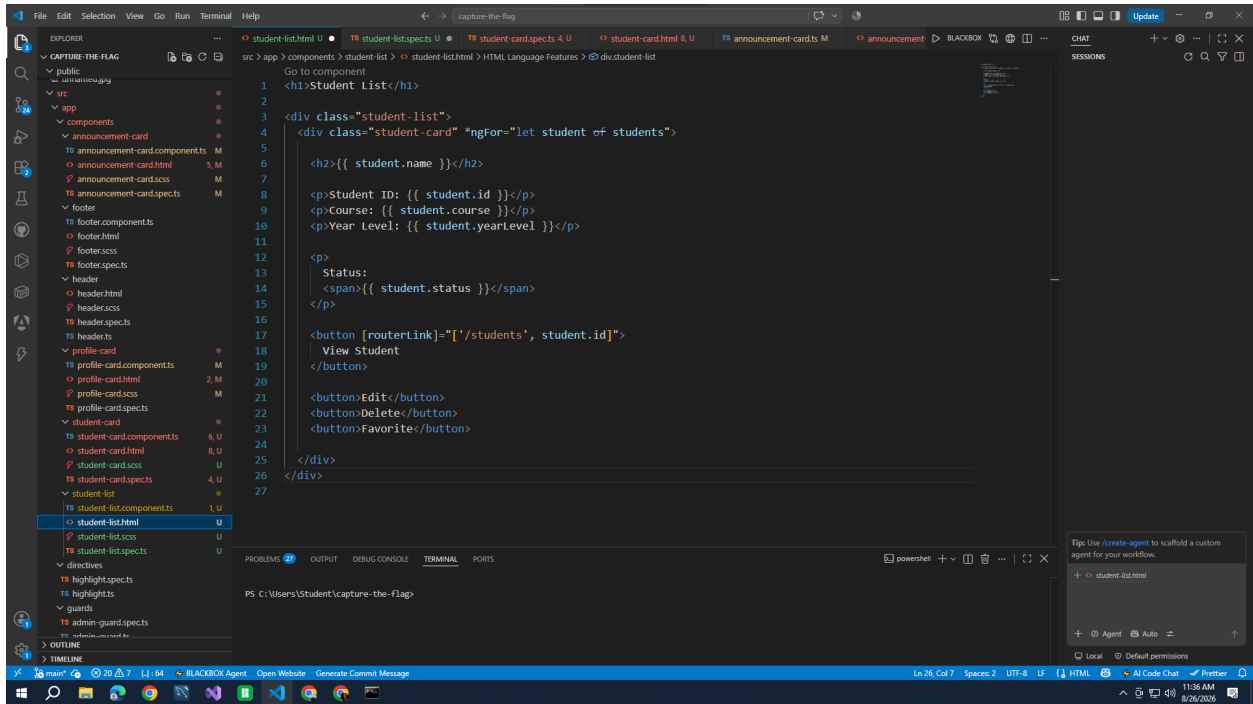
- Student cards
- Student information
- Active/Inactive status
- Existing actions

The Student List page will become the **main page** of the Student Portal.



Screenshot:

- Your Student List page displayed in the browser.



Flag 3 — Create the Student Detail Page

Objective: Create a separate page for viewing a student.

Create a new component for the Student Detail page.

For now, the page can display information such as:

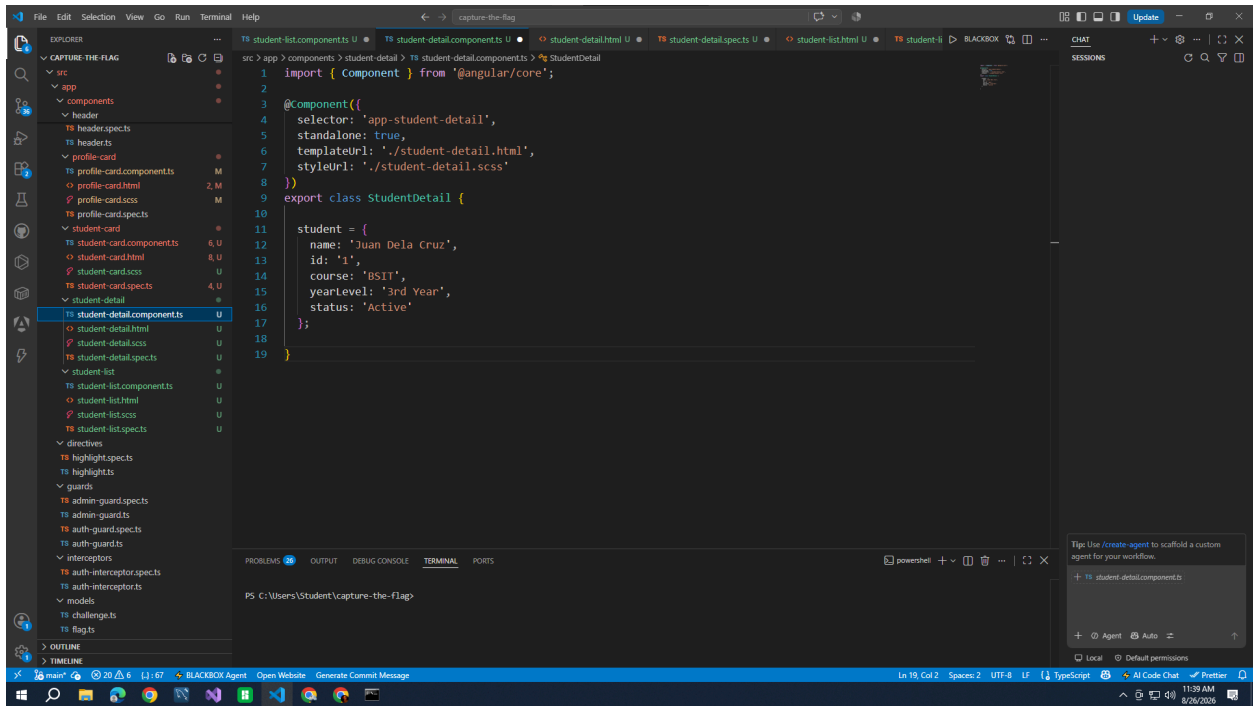
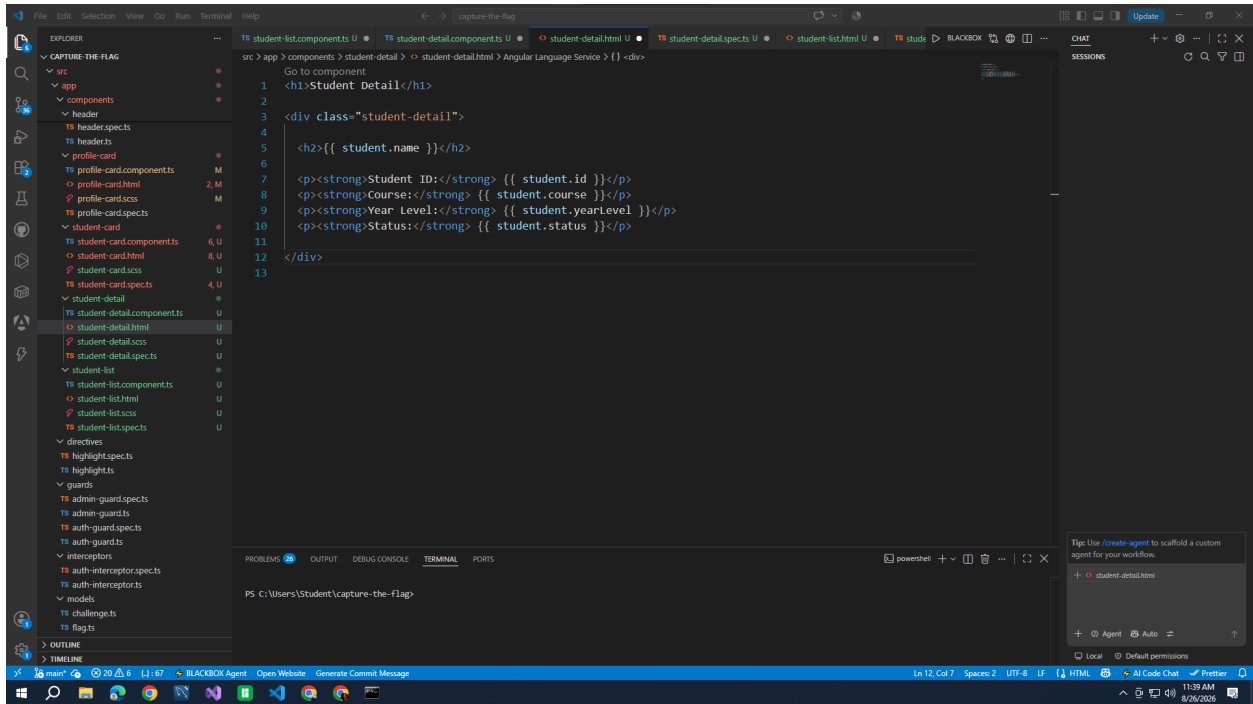
- Student name
- Student ID
- Course
- Year level
- Status

You will later connect this page to a specific student.



Screenshot:

- Your Student Detail page displayed separately from the Student List.



Flag 4 — Create the Add Student Page

Objective: Create a dedicated page for adding students.

Create another component for:

Add Student

For this challenge, **do not create any form fields yet**.

The page can simply display:

Add Student

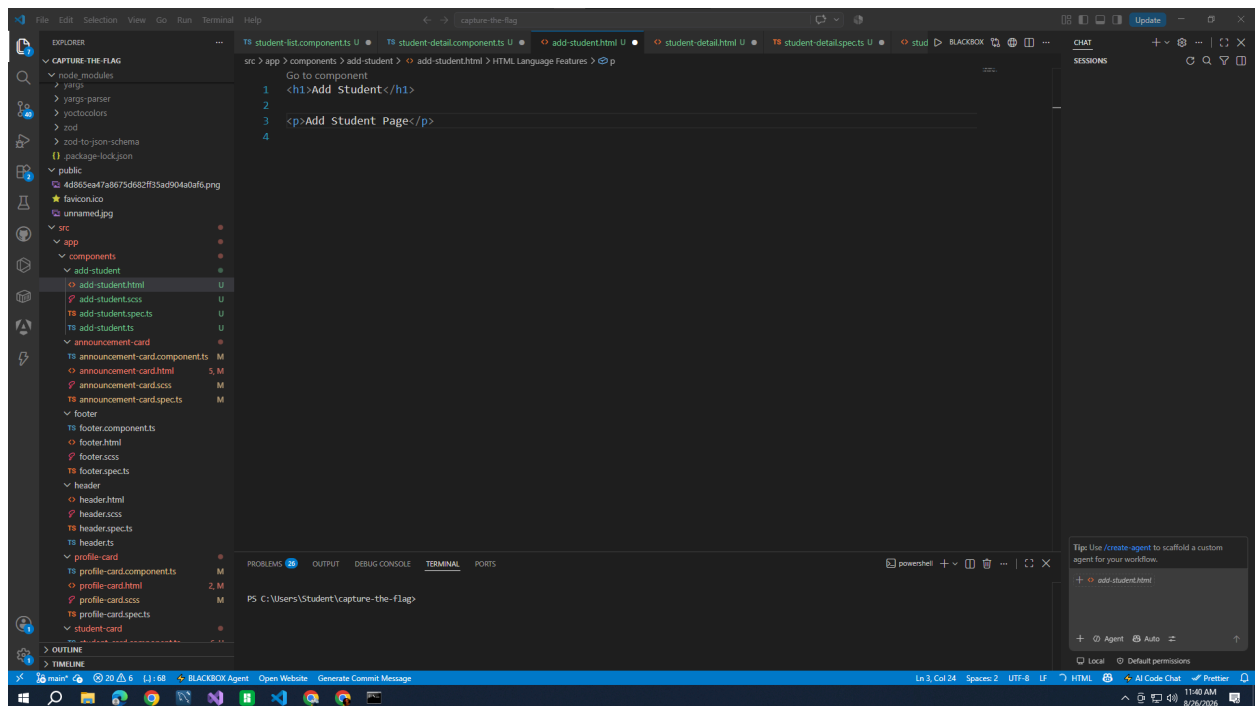
or

Add Student Page



Screenshot:

- The blank Add Student page displayed in the browser.



Flag 5 — Open the Portal Gate

Objective: Configure Angular Router.

Add routes for your three pages.

Your application should have routes similar to:

/students

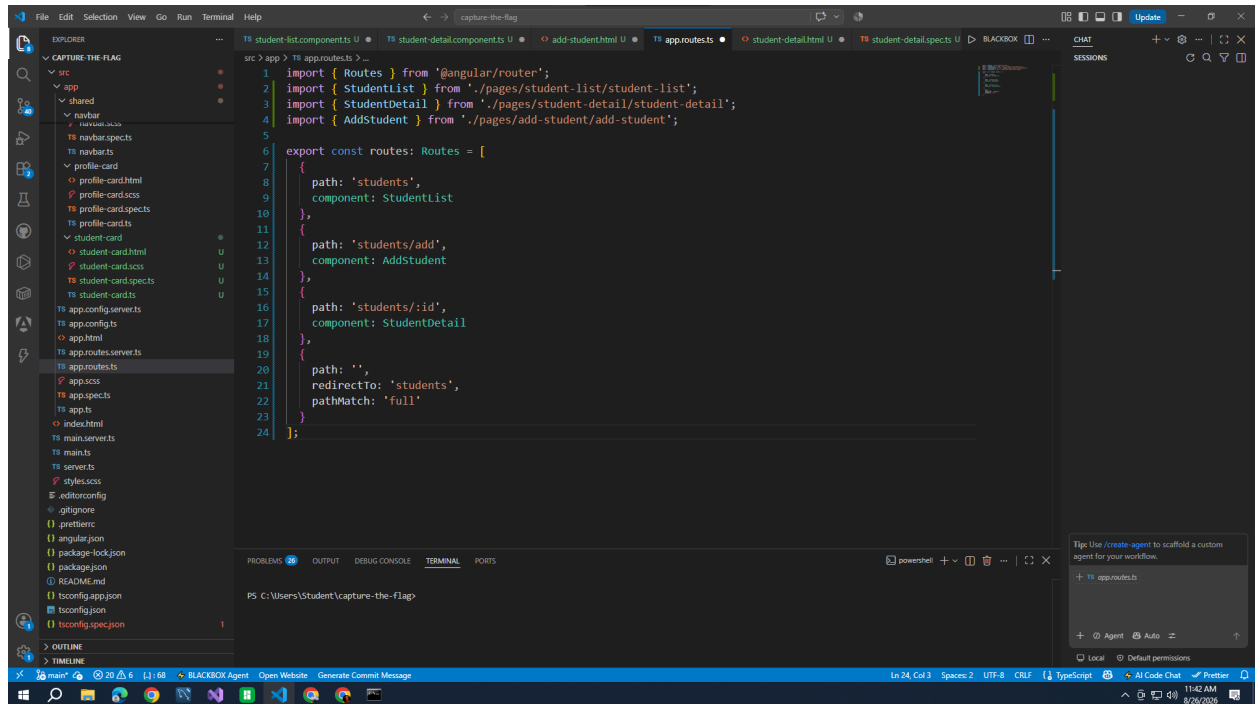
/students/:id

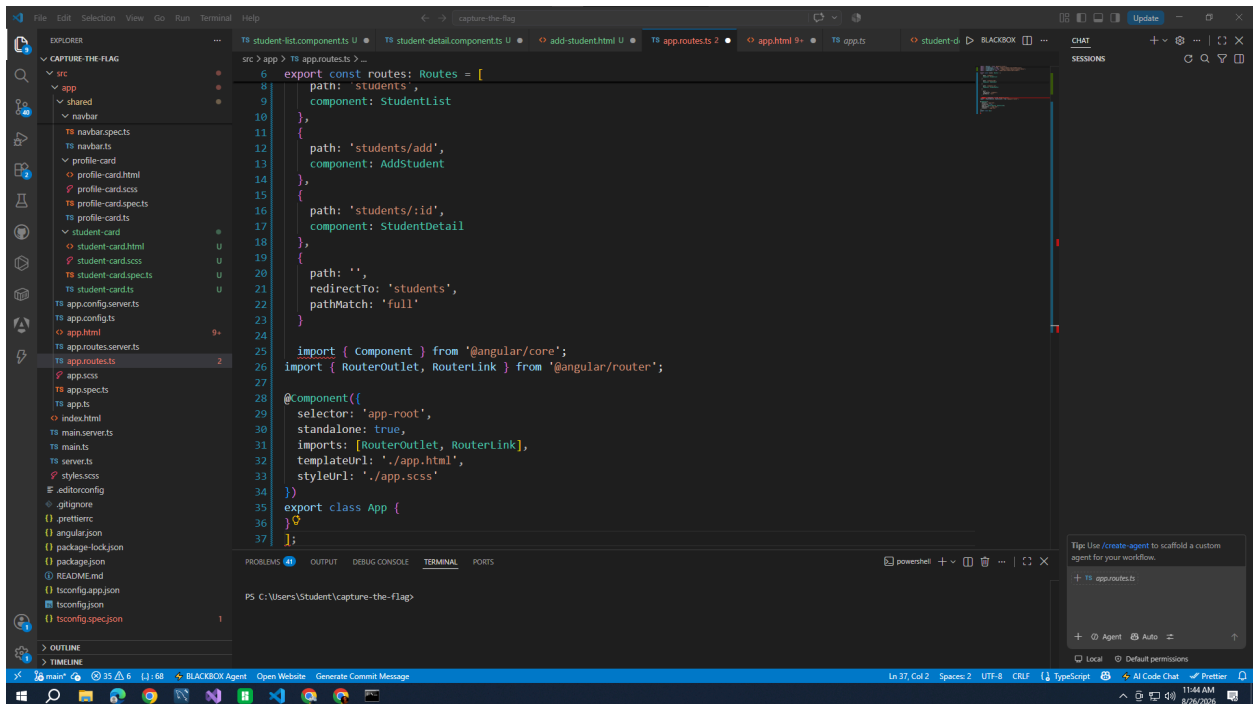
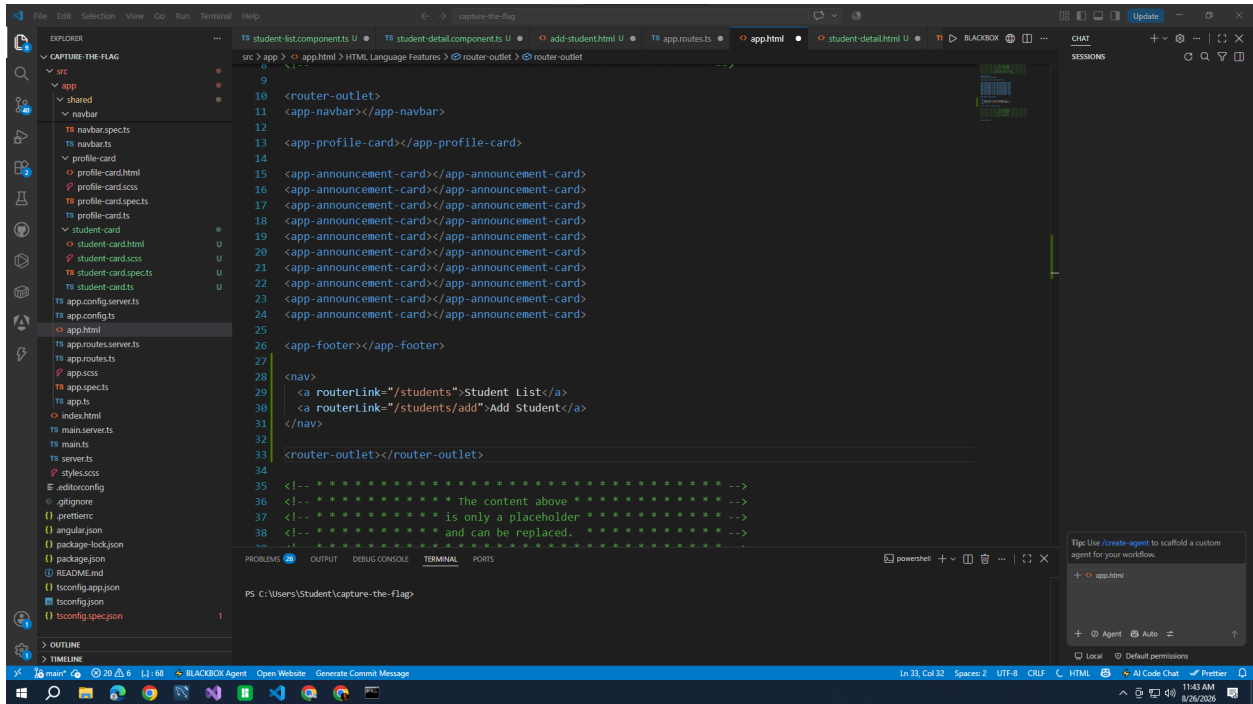
/students/add

The exact route configuration may vary depending on your project structure.

Screenshot:

- Your route configuration showing the three routes.





Flag 6 — Navigate Without Reloading

Objective: Add navigation between pages.

Create navigation links or buttons that allow the user to move between:

- Student List
- Add Student
- Student Detail

Use Angular's router navigation instead of manually changing the browser URL.

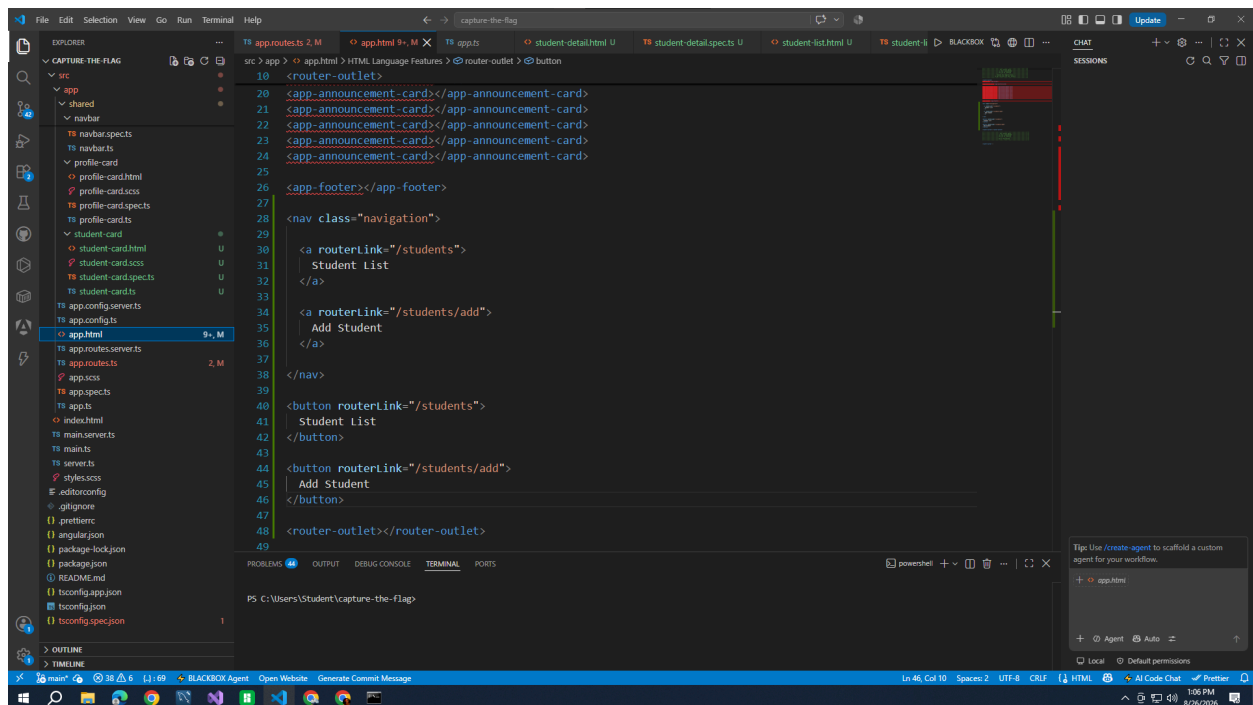
Hint:

routerLink



Screenshot:

- Your navigation menu/buttons.
- The browser changing between pages without a full page reload.



Flag 7 — Give Every Student a Page

Objective: Navigate from a student card to its detail page.

Add a button or link to each student card.

Example:

View Student

When clicked, it should navigate to that student's detail route.

Example:

`/students/1`

`/students/2`

`/students/3`

You don't need to load different student data yet. The goal is to successfully navigate using a route parameter.

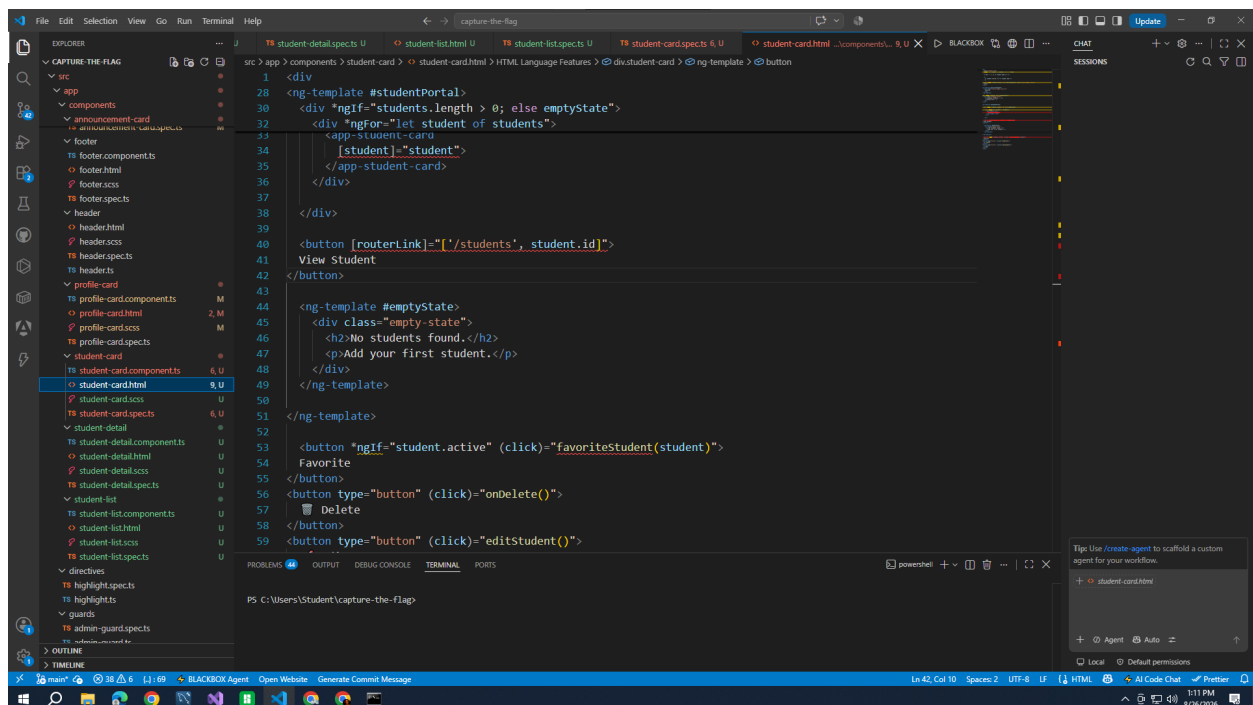
Hint:

`:id`



Screenshot:

- A student card with the View Student button.
- The resulting Student Detail URL/page.



Flag 8 — Build the Portal Navigation

Objective: Make the Student Portal feel like a multi-page application.

Your final navigation should allow users to move between:

Student List



Student Detail

Student List

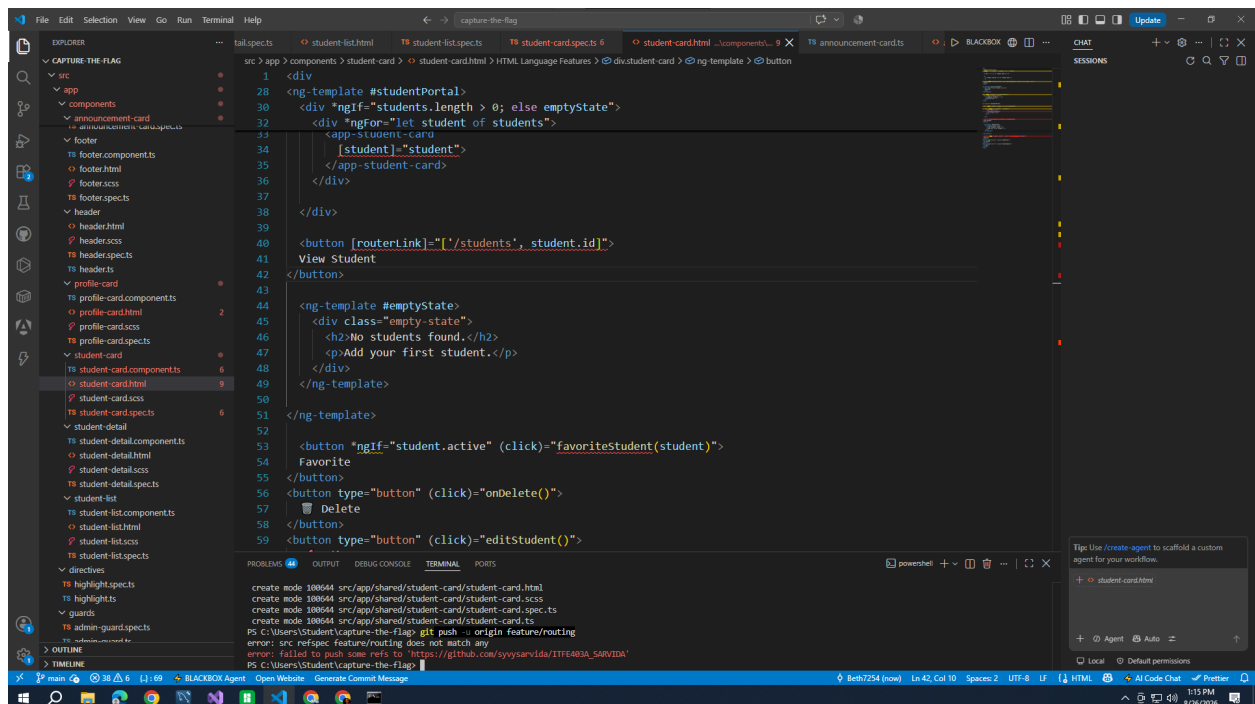


Add Student

The Student List should remain accessible after navigating to other pages.

 **Screenshot:**

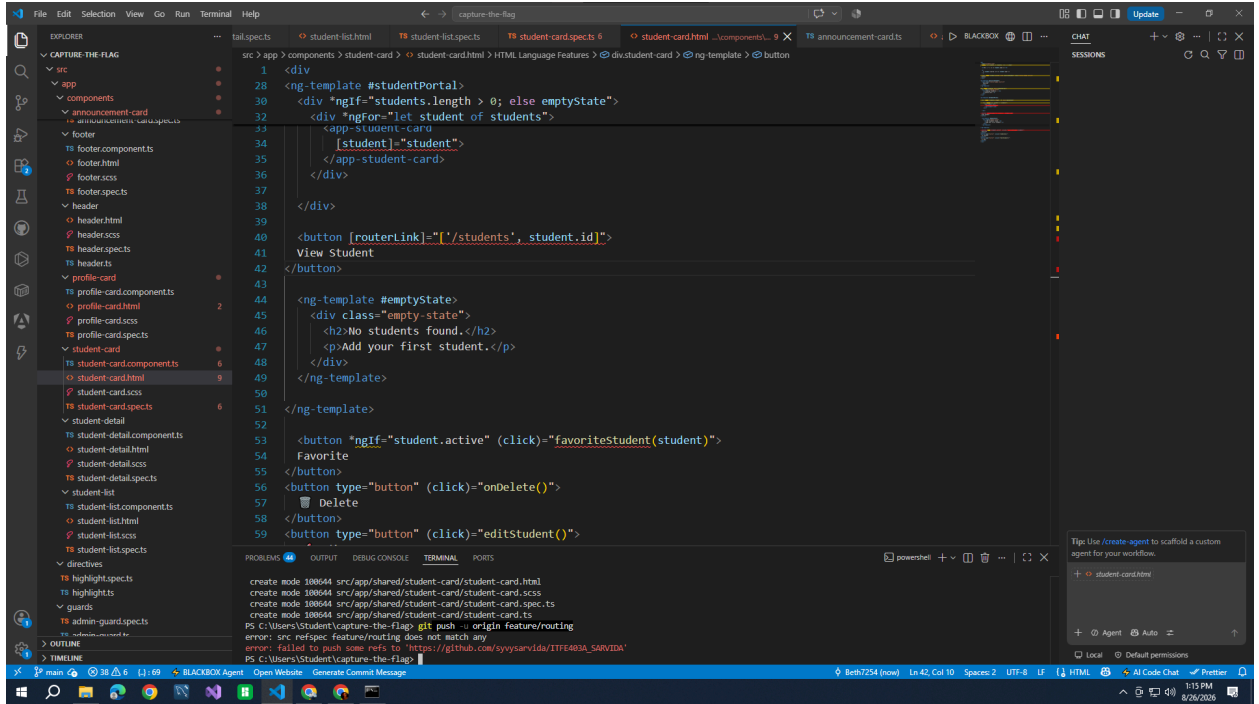
- Your complete navigation working between all three pages.



Flag 9 — Commit Your Progress

Final Flag Screenshot

- Successful push to **feature/routing**.



```
src > app > components > student-card > student-card.html > HTML Language Features > div.student-card > ng-template > button

1 <div>
28 <ng-template #studentPortal>
30 <div *ngIf="students.length > 0; else emptyState">
32 <div *ngFor="let student of students">
34 <app-student-card
35 [student]="student">
36 </app-student-card>
37 </div>
38 </div>
39 <button [routerLink]="['/students', student.id]">
40 View Student
41 </button>
42 </ng-template>
43 <ng-template #emptyState>
44 <div class="empty-state">
45 <h2>No students found.</h2>
46 <p>Add your first student.</p>
47 </div>
48 </ng-template>
49 </div>
50 </ng-template>
51 </div>
52 <button *ngIf="student.active" (click)="favoriteStudent(student)">
53 Favorite
54 </button>
55 <button type="button" (click)="onDelete()">
56 Delete
57 </button>
58 <button type="button" (click)="editStudent()">
59 </button>
60 </div>
61 </div>
62 </div>
63 </div>
64 </div>
65 </div>
66 </div>
67 </div>
68 </div>
69 </div>
70 </div>
71 </div>
72 </div>
73 </div>
74 </div>
75 </div>
76 </div>
77 </div>
78 </div>
79 </div>
80 </div>
81 </div>
82 </div>
83 </div>
84 </div>
85 </div>
86 </div>
87 </div>
88 </div>
89 </div>
90 </div>
91 </div>
92 </div>
93 </div>
94 </div>
95 </div>
96 </div>
97 </div>
98 </div>
99 </div>
100 </div>
```

```
create mode 100644 src/app/shared/student-card/student-card.html
create mode 100644 src/app/shared/student-card/student-card.scss
create mode 100644 src/app/shared/student-card/student-card.spec.ts
create mode 100644 src/app/shared/student-card/student-card.ts
PS C:\Users\Student\capture-the-flag> git push -u origin feature/routing
error: src refspec feature/routing does not match any
error: failed to push some refs to 'https://github.com/zyyysarv/ida/117f480a_sarv/ida/'
PS C:\Users\Student\capture-the-flag>
```